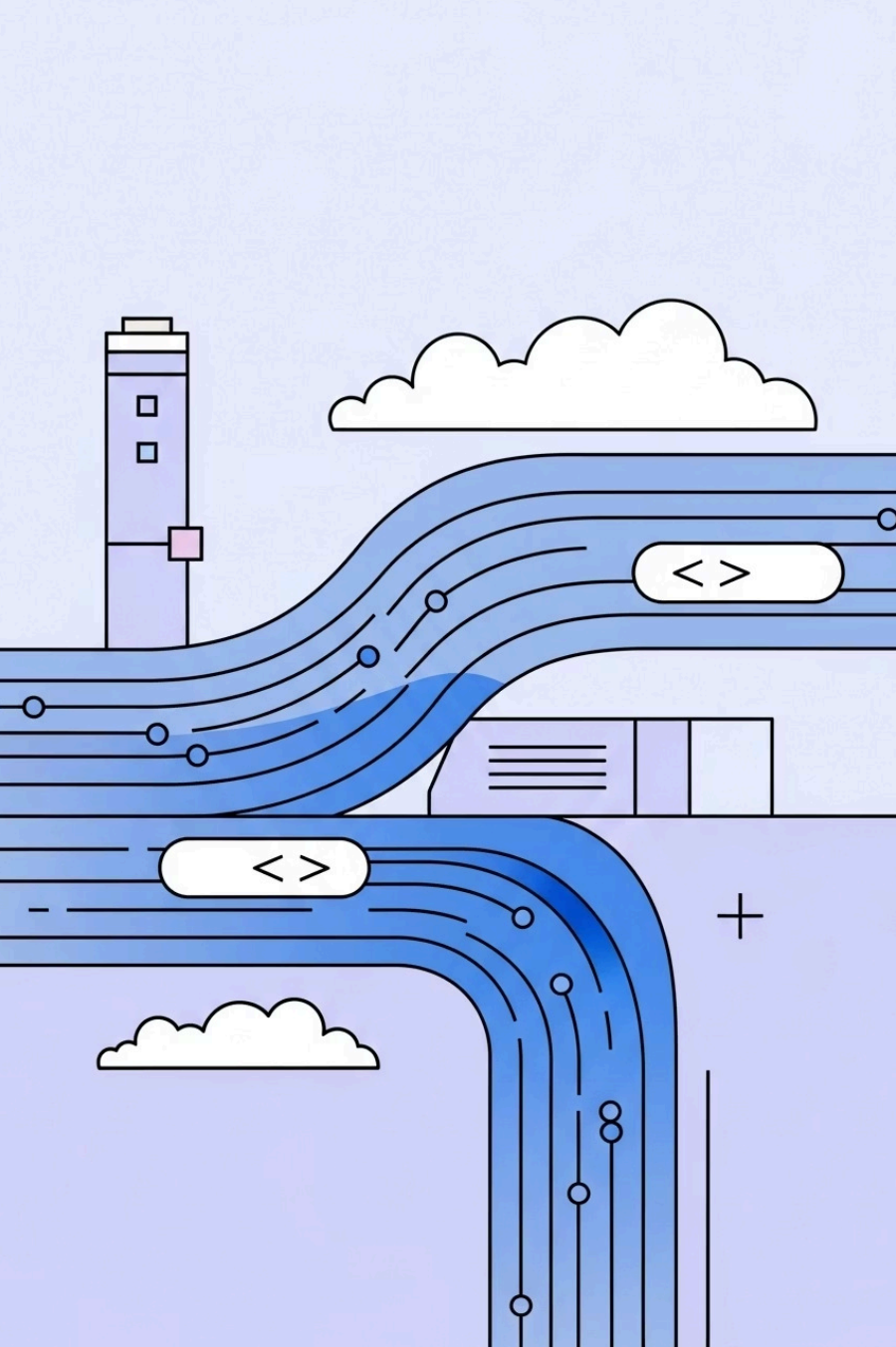




# Introducción a la Programación Funcional en Java con Streams

De código imperativo a código declarativo en Java

Una nueva forma de pensar sobre el código que transformará cómo trabajas con colecciones de datos en Java

# ¿Qué es la Programación Funcional?

La programación funcional es un **paradigma** que trata la computación como la evaluación de **funciones matemáticas**, evitando el cambio de estado y datos mutables.

En lugar de describir *cómo* hacer algo paso a paso, describes *qué* quieres obtener.

## Características principales

- Funciones como ciudadanos de primera clase
- Inmutabilidad de datos
- Evaluación perezosa (lazy)
- Composición de funciones

En Java 8+ podemos aplicar estos conceptos gracias a las interfaces funcionales, lambdas y streams.

# Imperativo vs Funcional

Dos formas diferentes de resolver el mismo problema:

1

## Enfoque Imperativo

- Se enfoca en **cómo** hacer las cosas
- Usa instrucciones paso a paso
- Modifica variables (estado mutable)
- Usa bucles y condicionales explícitos
- Requiere gestionar índices y contadores

2

## Enfoque Funcional

- Se enfoca en **qué** resultado queremos
- Usa transformaciones de datos
- Evita cambiar el estado (inmutabilidad)
- Usa operaciones de alto nivel
- Elimina la necesidad de variables temporales



# Ventajas de la Programación Funcional en Java



## Mayor legibilidad

El código se vuelve más expresivo y cercano al dominio del problema. Las intenciones quedan más claras al expresar el "qué" en lugar del "cómo".



## Menos código

Reduces significativamente la cantidad de código necesario para operaciones comunes como filtrado, mapeo y reducción de colecciones.



## Paralelismo sencillo

Al evitar estados compartidos, es mucho más fácil convertir operaciones secuenciales en paralelas (a veces con solo cambiar `stream()` por `parallelStream()`).



## Menos errores

La inmutabilidad y la ausencia de efectos secundarios reducen drásticamente los errores relacionados con el estado compartido y las condiciones de carrera.

# Comparación de Código: Imperativo vs Funcional

Veamos cómo filtrar números pares y ordenarlos de una lista:

## Enfoque Imperativo

```
List<Integer> numeros = Arrays.asList(5, 2, 8, 1, 7, 4);
List<Integer> pares = new ArrayList<>();

// Filtrar números pares
for (Integer num : numeros) {
    if (num % 2 == 0) {
        pares.add(num);
    }
}

// Ordenar la lista
Collections.sort(pares);

// Imprimir resultados
for (Integer num : pares) {
    System.out.println(num);
}
```

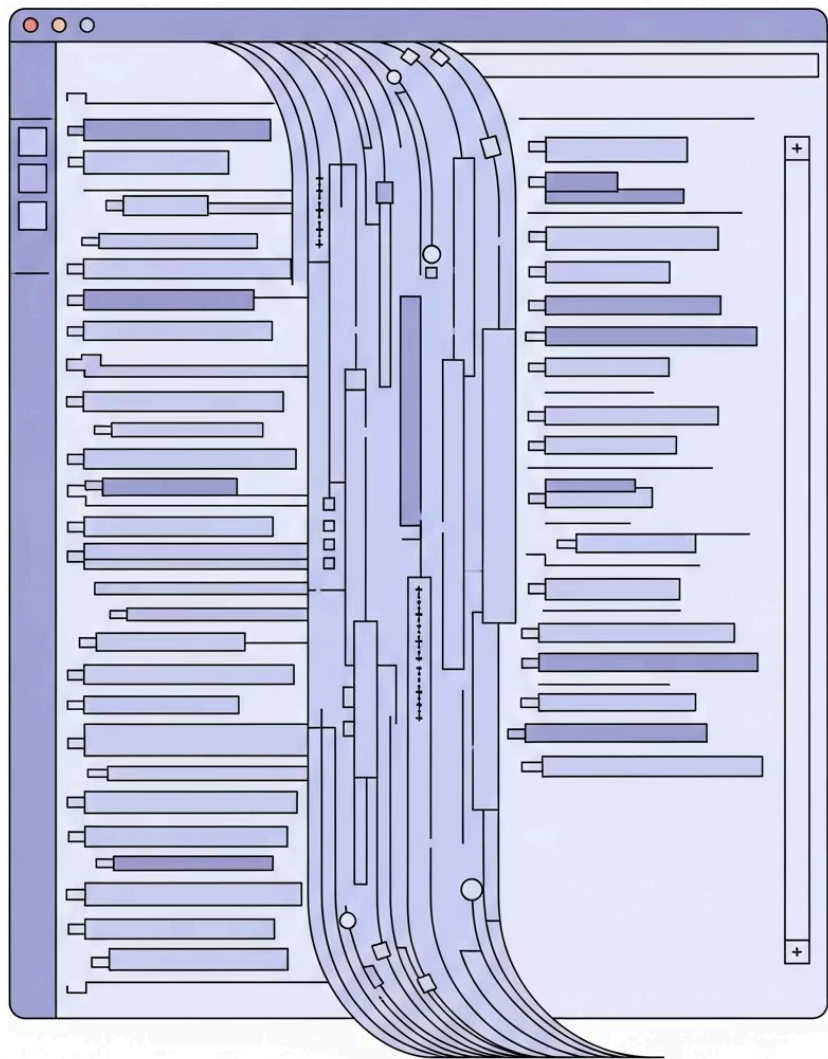
## Enfoque Funcional

```
List<Integer> numeros = Arrays.asList(5, 2, 8, 1, 7, 4);

numeros.stream()
    .filter(num -> num % 2 == 0)
    .sorted()
    .forEach(System.out::println);
```

**¡4 líneas vs 13 líneas!**

Más conciso, más legible y expresa claramente la intención: *filtrar, ordenar y mostrar*.



# ¿Qué Veremos a Continuación?

$$\frac{f}{dx}$$

## Expresiones Lambda

Sintaxis y funcionamiento de las funciones anónimas en Java

$((|))$

## API Stream

Operaciones intermedias y terminales para manipulación de datos

$\begin{matrix} \text{⓪} & \text{⓪} \\ \text{⓪} & \text{⓪} \end{matrix}$

## Ejemplos Prácticos

Casos de uso comunes y patrones de diseño funcional